8-3 · 수집→Lake→Spark→VectorDB 흐름 자동화 · 실습 1.5H

수집→Lake→변환 태스크 구현

7주차 셋을 붙이고, 그 위에 네 태스크

수도권ICT 이노베이션스퀘어

데이터 엔지니어링 & RAG 파이프라인 160H · 3기

이 시간의 목표

1.5시간 뒤에는

① qdrant · rag-minio · ollama 를 Airflow 네트워크에 붙여 이름으로 부른다

② .env 네 줄로 패키지와 주소를 Airflow 컨테이너에 넘긴다

③ 7주차 build_kb.py 를 부르는 rag_pipeline 을 돌려 lecture_kb 11

④ 짧은 문서를 일부러 넣어 validate 가 아래를 막는 것을 본다

사전 준비: 실습점검 1~3단계(컨테이너 셋 · lecture_kb 10 · Airflow healthy · af)

실습 흐름

일곱 단계 — ~/airflow-lab 에서 끝까지

2단계(이름으로 부르기)를 통과하지 못하면 뒤가 전부 실패합니다 — 반드시 먼저

1·2. 네트워크	3. 패키지·주소	4. 모듈·문서	5·6. DAG·실행	7. 일부러 실패
network connect 이름으로 부르기	.env 네 줄 up -d 약 4분	7주차 셋 복사 airflow_ops.txt	rag_pipeline count 11	bad.txt → validate failed

1단계

7주차 컨테이너를 Airflow 네트워크에 붙이세요

1. **docker network connect airflow-lab_default qdrant**
   rag-minio · ollama 도 같은 명령 — 지우거나 옮기지 않는다

2. **docker network inspect airflow-lab_default ...**
   airflow-lab-... 일곱과 ollama · qdrant · rag-minio 가 보이면 된다

3. **endpoint with name ... already exists**
   이미 붙어 있다는 뜻 — 괜찮다(실측)

그대로 남는 것

모델 · 지식

베이스 · 문서

2단계

Airflow 안에서 이름으로 불러 보세요

worker 안에서 부른 곳	결과(실측)
qdrant:6333/collections	lecture_kb · lecture_kb_200 · lecture_notes · my_notes
rag-minio:9000 /minio/health/live	minio 200
ollama:11434	Ollama is running
Could not resolve host	1단계 연결이 빠진 것

docker compose exec airflow-worker bash -c 'curl ...' — 여기서 막히면 뒤가 전부 실패한다

3단계

패키지와 주소를 .env 로 넘기세요

1. **_PIP_ADDITIONAL_REQUIREMENTS=qdrant-client minio ...**
   python-dotenv · langchain-ollama · langchain-openai · langchain-text-splitters

2. **QDRANT_URL=http://qdrant:6333**
   7주차 모듈은 주소가 없으면 localhost 를 쓴다

3. **MINIO_ENDPOINT=rag-minio:9000 · OLLAMA_HOST=http://ollama:11434**
   cat >> .env 로 끝에 덧붙인다 — 공식 compose 의 env_file

4. **OpenAI 경로만 OPENAI_API_KEY 한 줄**
   Ollama 경로면 건너뛴다 — .env 는 Git 에 올리지 않는다

3단계 · 반영

up -d 로 컨테이너를 새로 만드세요

1. **docker compose up -d → sleep 240**
   컨테이너마다 패키지를 설치한다 — 실측 239초

2. **docker compose ps — 일곱 줄 (healthy)**
   cat .env 는 여섯 줄(AIRFLOW_UID · FERNET_KEY · 넷)

3. **exec airflow-worker python -c "import qdrant_client ..."**
   ok http://qdrant:6333 http://ollama:11434 (실측)

restart 는

반영되지 않는다

> _PIP_ADDITIONAL_REQUIREMENTS 는 학습용 — 운영은 Dockerfile 로 이미지를 따로 만든다

4단계

7주차 모듈과 새 문서를 넣으세요

1. **cp ~/data-eng/week7/rag_env.py ... build_kb.py dags/**
   새로 쓰지 않는다 — dags/ 는 파이썬 경로라 import build_kb 로 바로

2. **mkdir -p dags/data/incoming**
   투입구 — 지식베이스에 아직 없는 문서를 둔다

3. **cat > dags/data/incoming/airflow_ops.txt**
   「Airflow 운영 수칙」 세 줄짜리 새 문서

고칠 곳

없음 — .env 가 주소

> 세 모듈은 「airflow」「dag」 낱말이 없어 dag-processor 가 DAG 로 읽지 않는다

5단계

rag_pipeline — 네 태스크

collect	validate	build_index	report
투입구 .txt → raw-docs/incoming/	20자 미만이면 raise ValueError	import build_kb build_kb.main()	lecture_kb 조각 수 출력

from airflow.sdk import DAG · schedule=None · catchup=False · retries 1 · retry_delay 30초

collect 는 같은 키에 덮어쓰고 build_index 는 문서마다 지우고 다시 넣는다 — 몇 번 돌려도 같다(멱등)

5단계 · 핵심 줄

파일을 쓰고 바로 읽히게 하세요

Operator	from airflow.providers.standard.operators.python import PythonOperator
배선	t_collect >> t_validate >> t_index >> t_report
바로 읽히게	af dags reserialize — 기다리면 최대 5분
임포트 오류	af dags list-import-errors → No data found 면 통과

무거운 import 는 함수 안에 — 최상위에 두면 dag-processor 가 파일을 읽을 때마다(30초마다) 접속을 만든다

6단계

돌려 보세요 — 10 에서 11 로

1. **af dags unpause rag_pipeline → af dags trigger rag_pipeline**
   sleep 60 뒤 상태를 본다

2. **af tasks states-for-dag-run rag_pipeline "$RUN" -o plain**
   collect · validate · build_index · report 네 줄 success

3. **lecture_kb points/count → 11**
   10 에 airflow_ops.txt 한 조각(실측)

4. **첫 실행 build_index 약 27초 · 전체 약 40초**
   임베딩 모델을 올리느라 — 두 번째부터 build_index 약 5초

실측

count

11

7단계 — 일부러 실패시키기

검증이 실제로 막는지 — 「짧음」을 넣어 본다

검사기는 일부러 깨뜨려 봐야 동작한다고 말할 수 있습니다

1. bad.txt	2. 다시 trigger	3. validate failed	4. 아래는 막힘	5. 되돌리기
echo "짧음" > incoming/bad.txt	af dags trigger rag_pipeline	너무 짧은 문서: incoming/bad.txt	build_index · report upstream_failed	rm bad.txt → trigger → 네 줄 success

확인 포인트

이 1.5시간이 끝날 때 확인할 것

1. worker 안에서 qdrant · rag-minio · ollama 가 이름으로 닿는다
2. import 검사가 ok http://qdrant:6333 http://ollama:11434 를 낸다
3. 네 태스크가 success 이고 lecture_kb 가 11 이다
4. bad.txt 를 넣으면 validate failed · 아래는 upstream_failed
5. bad.txt 를 지우고 돌리면 collect 가 MinIO 에서도 지운다

validate 는 retries 1 — 60초 전에 보면 up_for_retry 일 수 있습니다

8-3 · 실습 1.5H 마무리

다음은 임베딩→VectorDB 적재와 전체 DAG 완성 (1H)

두 번 돌려 멱등을 보고, 지운 문서의 조각을 prune 으로 정리한 뒤 @daily 를 붙입니다

- ✓ 두 번 돌려도 count 11
- ✓ 투입구에서 지운 문서의 조각이 남는가
- ✓ prune 태스크로 11 → 10
- ✓ @daily · next-execution

가져갈 것

success 네 줄 · lecture_kb 11